

Relaciones entre tablas

Las bases de datos relacionales se construyen sobre la idea de vincular tablas para evitar la redundancia de datos y mantener su integridad. En lugar de almacenar toda la información en una sola tabla grande, se divide en tablas más pequeñas y lógicas que se conectan entre sí.

Tipos de Relaciones

Existen tres tipos principales de relaciones que se establecen a través de claves primarias y foráneas:

- **Relación Uno a Uno (1:1):** Cada registro en una tabla está vinculado exactamente a un registro en otra tabla. Por ejemplo, una tabla de *Usuarios* y una tabla de *Perfiles*. Cada usuario tiene un solo perfil y cada perfil pertenece a un solo usuario.
- **Relación Uno a Muchos (1:N):** Un registro en la primera tabla puede estar relacionado con múltiples registros en la segunda tabla. Es la relación más común. Por ejemplo, una tabla de *Usuarios* y una tabla de *Transacciones*. Un usuario puede tener muchas transacciones, pero cada transacción pertenece a un solo usuario.
- **Relación Muchos a Muchos (N:M):** Múltiples registros en una tabla pueden estar relacionados con múltiples registros en otra tabla. Por ejemplo, una tabla de *Estudiantes* y una tabla de *Clases*. Un estudiante puede estar en muchas clases y una clase puede tener muchos estudiantes. Para implementar esta relación, se necesita una tercera tabla, llamada tabla intermedia o "pivot", que almacena las claves foráneas de las otras dos tablas.

Relaciones en Laravel con Eloquent

Para definir las relaciones se deben agregar nuevos métodos en cada modelo, que determinan a qué modelo puntual se encuentra relacionado. Cada método creado tendrá el nombre de la clase a la cual está relacionado y un método de relación de Eloquent que dependerá del tipo de relación a crear.

1. Relación Uno a Muchos (One to Many)

Esta relación es la más común. Por ejemplo, un User puede tener muchos Posts.

- **En el modelo "uno" (User):** Usamos el método `hasMany()` para indicar que un User tiene muchos Posts.

```
// Importamos el namespace del tipo de relación a usar
use Illuminate\Database\Eloquent\Relations\HasMany;

// Declaramos la relación mediante un nuevo método dentro de la clase
public function posts(): HasMany
{
```

```

        return $this->hasMany(Post::class);
    }

```

- **En el modelo "muchos" (Post):** Usamos el método `belongsTo()` para indicar que un Post pertenece a un User.

```

use Illuminate\Database\Eloquent\Relations\BelongsTo;

public function user(): BelongsTo
{
    return $this->belongsTo(User::class);
}

```

¿Cómo sabemos en cual modelo va el método *hasMany* y en cual el método *belongsTo*? Para ello seguiremos la siguiente regla:

- El método **`belongsTo()`** siempre va en el modelo que **contiene la clave foránea**. Es el modelo "hijo" o el que "pertenece a". Por ejemplo, en una relación de User a Post, la clave foránea `user_id` está en la tabla posts. Por lo tanto, el método `belongsTo()` va en el modelo Post.
- El método **`hasMany()`** siempre va en el modelo que **no tiene la clave foránea**, pero que tiene "muchos" de algo. Es el modelo "padre" o el que "tiene muchos". Siguiendo el ejemplo anterior, el modelo User es el que "tiene muchas" publicaciones, por lo que el método `hasMany()` va en el modelo User.

2. Relación Uno a Uno (One to One)

Similar a la relación uno a muchos, pero con una diferencia clave: ambas tablas solo tienen una relación con la otra. Por ejemplo, un User tiene un solo Profile.

- **En el modelo "uno" (User):** Usamos el método `hasOne()`.

```

use Illuminate\Database\Eloquent\Relations\HasOne;

public function profile(): HasOne
{
    return $this->hasOne(Profile::class);
}

```

- **En el modelo "uno" (Profile):** Usamos el método `belongsTo()`, al igual que en la relación uno a muchos.

```

use Illuminate\Database\Eloquent\Relations\BelongsTo;

public function user(): BelongsTo
{
    return $this->belongsTo(User::class);
}

```

```
}
```

Nuevamente, aplicamos la siguiente regla:

- El método **belongsToMany()** va en el modelo que **contiene la clave foránea**.
- El método **hasOne()** va en el modelo que **no tiene la clave foránea**.

3. Relación Muchos a Muchos (Many to Many)

Esta relación es más compleja y requiere una **tabla pivote** o intermedia. Por ejemplo, un User puede tener muchos Roles, y un Role puede pertenecer a muchos Users.

- **En el modelo "muchos" (User):** Usamos el método `belongsToMany()`.

```
use Illuminate\Database\Eloquent\Relations\BelongsToMany;

public function roles(): BelongsToMany
{
    return $this->belongsToMany(Role::class);
}
```

- **En el modelo "muchos" (Role):** Usamos el mismo método `belongsToMany()`.

```
use Illuminate\Database\Eloquent\Relations\BelongsToMany;

public function users(): BelongsToMany
{
    return $this->belongsToMany(User::class);
}
```

Tabla Pivote: Laravel espera una tabla intermedia con un nombre en singular y en orden alfabético, como `role_user`, que contenga las claves foráneas `user_id` y `role_id`.

```
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('role_user', function (Blueprint $table) {
            $table->foreignId('user_id')->constrained();
            $table->foreignId('role_id')->constrained();
        });
    }
}
```

```

    }

    public function down(): void
    {
        Schema::dropIfExists('role_user');
    }
};

```

Convenciones de nombres para crear relaciones

Para que las relaciones se implementen de la forma correcta, Laravel espera que se cumplan las siguientes convenciones en cuanto al nombre de los elementos:

1. Las tablas se deben llamar en plural. Ejemplo: *users*, *posts*.
2. Los modelos deben llamarse en la forma singular de su respectiva tabla. Ejemplo: *User*, *Post*.
3. La clave primaria de cada tabla debe llamarse *id*.
4. Las claves foráneas deben nombrarse según la tabla relacionada de forma singular seguido de un guión bajo y la palabra *id*. Por ejemplo: *post_id*, *user_id*.
5. Las tablas pivot (que representan relaciones muchos a muchos) deben nombrarse utilizando la palabra singular de cada una de las tablas relacionadas y en orden alfabético. Por ejemplo: la tabla que relaciona *posts* y *users* se debe llamar *post_user*.

Si no se cumple con algunas de estas convenciones, debemos indicárselo a Eloquent en los modelos. Ya sabemos como sobrescribir los nombres de tablas y de sus ID primarios (Ver apunte de clase pasada), ahora veamos como indicar las relaciones cuando no se cumplen las convenciones.

Relaciones con claves foráneas diferentes

Si el nombre de la clave foránea no respeta la convención del punto 4 visto anteriormente, podemos indicar el nombre de la clave como segundo argumento en el método de relación. Retomando el ejemplo de *User* y *Post*, ¿cómo indicaríamos la relación si la tabla *posts* tuviera una clave foránea llamada *autor_id* en lugar de *user_id*?

En el modelo **Post**:

```

public function autor(): BelongsTo
{
    return $this->belongsTo(User::class, 'autor_id');
}

```

En el modelo **User**:

```

public function posts(): HasMany

```

```
{
    return $this->hasMany(Post::class, 'autor_id');
}
```

Relaciones muchos a muchos con tabla pivot no convencional

Si el nombre de la tabla pivot o de sus claves foráneas no respetan la convención vista previamente, podemos especificarlos como argumentos en el método `belongsToMany` de cada modelo.

Supongamos que la tabla pivot de la relación entre *users* y *roles* se llamase *usuario_rol* y que las claves foráneas se llamasen *id_usuario* y *id_rol*. Para establecer la relación la definiríamos de la siguiente forma en el modelo **User**:

```
public function roles(): BelongsToMany
{
    return $this->belongsToMany(Role::class, 'usuario_rol',
    'id_usuario', 'id_rol');

    // Argumentos:
    // 1. Clase del modelo relacionado (Role::class)
    // 2. Nombre de la tabla pivote ('usuario_rol')
    // 3. Clave foránea del modelo local ('id_usuario')
    // 4. Clave foránea del modelo relacionado ('id_rol')
}
```

Ejemplo: crear categorías de transacciones

Agreguemos un ejemplo a nuestro sistema: la tabla de categorías que representa una categoría específica relacionada a cada transacción.

1) Crear la migración para categorías

Creemos la migración con el siguiente comando:

```
php artisan make:migration create_categorias_migration
```

Luego modificamos el método **up()** de la migración para incluir la estructura de la tabla:

```
public function up(): void
{
    Schema::create('categorias', function (Blueprint $table) {
        $table->id();
    });
}
```

```

        $table->string('nombre');
        $table->enum('tipo', ['ingreso', 'gasto']);
        $table->timestamps();
    });
}

```

2) Modificar la migración de transacciones

Debemos agregar a la tabla de transacciones el id de categoría al cual pertenece. Para esto podemos tomar dos decisiones:

- Podemos crear una nueva migración que solamente agregue el nuevo campo de *categoria_id* a transacciones, de modo que podamos ejecutarla sin la necesidad de refrescar toda la base de datos.
- Podemos modificar la migración ya existente de transacciones, con lo cual deberemos refrescar todas las migraciones existentes. El detalle a tener en cuenta es que la migración de categorías deberá ejecutarse si o si primero que la de transacciones, para lo cual podemos modificar el nombre de cualquiera de los dos archivos.

Optaremos por la segunda opción. Abrimos el archivo de migración de transacciones y modificamos el método **up()** de la siguiente forma:

```

Schema::create('transacciones', function (Blueprint $table) {
    $table->id();
    $table->string('descripcion');
    $table->decimal('monto', 10, 2);
    $table->date('fecha_transaccion');
    $table->foreignId('categoria_id')->constrained();
    $table->timestamps();
    $table->softDeletes();
});

```

Usamos el método *foreignId()* para indicar que ese campo también corresponde con una clave foránea y el método *constrained()* para indicar que está relacionado con el campo *id* de la tabla *categorías*. Si no seguimos las convenciones, deberemos reemplazar *constrained()* por la siguiente sintaxis:

```

$table->foreign('categoria_id')->references('idcategoria')-
->on('categorias_transacciones');

```

- **references:** define a qué clave primaria está vinculada la clave foránea. En este caso, en la otra tabla la PK se llama *idcategoria*.
- **on:** define el nombre de la tabla vinculada. En este caso, el nombre de la tabla es *categorías_transacciones*.

3) Ejecutar las migraciones

Vamos a migrar nuevamente toda la estructura de la base de datos para mostrar los cambios. Primero, renombramos el archivo de la migración de transacciones por una fecha y hora más actual, asegurándonos que se ejecutará antes de la migración de categorías.

Luego, ejecutamos una migración desde cero:

```
php artisan migrate:--fresh
```

4) Definir relaciones en los modelos

Creamos el modelo Categoría con la relación de la tabla transacciones:

```
<?php
namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\HasMany;

class Category extends Model
{
    public function transacciones(): HasMany
    {
        return $this->hasMany(Transaccion::class);
    }
}
```

Luego, modificamos el modelo Transaccion para agregar la nueva relación:

```
public function categoria(): BelongsTo
{
    return $this->belongsTo(Categoria::class);
}
```

Antes de avanzar, podemos agregar al menos una categoría de forma manual a la base de datos y agregar una o dos transacciones que utilicen dicha categoría.

5) Probar realizar una consulta de prueba

En una ruta de prueba, leemos la primera transacción de la base de datos y mediante la relación creada en el modelo obtenemos su categoría:

```
// Buscar una transacción
```

```
$transaccion = Transaccion::find(1);  
// Acceder a la categoría de la transacción  
$categoria = $transaccion->categoria;  
echo "Descripción: " . $transaccion->descripcion . PHP_EOL;  
echo "Categoría: " . $categoria->nombre . PHP_EOL;  
echo "Tipo: " . $categoria->tipo . PHP_EOL;
```

También podemos probar crear un registro de ambos modelos de la siguiente forma:

```
// Crear una nueva categoría usando el método create()  
$gasto = Categoria::create([  
    'nombre' => 'Gasto del mes',  
    'tipo' => 'gasto',  
]);  
// Crear una nueva transacción y vincularla a la categoría usando el  
método create()  
$transaccion = Transaccion::create([  
    'descripcion' => 'Pago de telefonía móvil',  
    'monto' => 20000,  
    'fecha_transaccion' => now(),  
    'categoria_id' => $gasto->id,  
]);
```

¿Y si deseamos crear una transacción, pero la categoría ya existe en la base de datos? Podemos hacer primero la consulta SELECT para obtener la categoría y luego vincularla en la creación de la transacción.

```
// Buscar la categoría 'Gasto del mes' en la base de datos  
$gasto = Category::where('nombre', 'Gasto del mes')->first();  
$transaccion = Transaccion::create([  
    'descripcion' => 'Pago de telefonía móvil',  
    'monto' => 20000,  
    'fecha_transaccion' => now(),  
    'categoria_id' => $gasto->id,  
]);
```